UNIVERSIDAD AUTÓNOMA TOMÁS FRÍAS

Carrera de Ingeniería Informática

INF781 — SEGURIDAD DE SOFTWARE

GUÍA DE LABORATORIO N° 1

Configuración Segura del Proyecto Base con Laravel

Proyecto: tasks-app | Stack: Laravel 12 + PostgreSQL

Materia:	INF781 Seguridad de Software	**Semestre:**	7° Semestre
Docente:	M.Sc. Huáscar F. Gonzales G.	**Gestión:**	2026
Duración:	5 horas académicas	**Modalidad:**	Presencial

🎯 **COMPETENCIA:** Configura un proyecto Laravel de forma segura aplicando principios de mínima exposición, gestión de secretos mediante variables de entorno y configuración robusta de la base de datos PostgreSQL, garantizando que el entorno base cumpla con estándares de seguridad en desarrollo web.

Prerrequisitos del Entorno

Software Requerido	Versión Mínima	Comando de Verificación	Resultado Esperado
PHP	8.2 o superior	`php --version`	*PHP 8.2.x (cli) o superior*
Composer	2.5 o superior	`composer --version`	*Composer version 2.5.x o superior*
PostgreSQL	15 o superior	`psql --version`	*psql (PostgreSQL) 15.x o superior*
Git	2.40 o superior	`git --version`	*git version 2.40.x o superior*
VS Code (recomendado)	Última estable	`code --version`	*1.90.x o superior*

■ 1. Marco Teórico: Seguridad en la Configuración de Proyectos Laravel

1.1 Por qué la Configuración Importa en Seguridad

La mayoría de brechas de seguridad en aplicaciones web no provienen de vulnerabilidades en el lenguaje de programación, sino de errores de configuración. Laravel, como framework moderno, ofrece mecanismos robustos para separar la configuración del código fuente, pero estos mecanismos solo funcionan si el desarrollador los aplica correctamente.

Los tres principios clave que guían esta guía son:

- Mínima exposición: no revelar información del sistema en producción.
- Separación de secretos: credenciales nunca en el repositorio de código.
- Fallar de forma segura: cuando algo sale mal, el sistema no debe revelar detalles internos.

1.2 El archivo .env y la Gestión de Secretos

Laravel utiliza el paquete vlucas/phpdotenv para cargar variables de entorno desde el archivo .env. Este archivo contiene información sensible como credenciales de base de datos, claves de API y la APP_KEY de cifrado. Las reglas de oro para el archivo .env son:

> ⚠️ **REGLA DE ORO:** El archivo .env JAMÁS debe ser agregado al repositorio de control de versiones (Git). Incluirlo equivale a publicar las credenciales de la aplicación. El .gitignore debe excluirlo explícitamente.

1.3 APP_KEY y Cifrado en Laravel

La APP_KEY es una clave de 32 caracteres que Laravel usa para cifrar cookies de sesión, tokens CSRF y datos en el cache. Si esta clave es comprometida o ausente, toda la seguridad del cifrado de la aplicación colapsa. Debe generarse una sola vez por entorno con:

```
php artisan key:generate
```

1.4 APP_DEBUG: La Amenaza del Stack Trace

Cuando APP_DEBUG=true, Laravel muestra stack traces completos ante errores. Estos traces revelan:

- Rutas absolutas del sistema de archivos del servidor.
- Variables de entorno (incluyendo credenciales).
- Versiones exactas de librerías y el framework.
- Lógica interna de la aplicación.

> 🔴 **CRÍTICO:** En producción, APP_DEBUG debe ser siempre false. En desarrollo puede ser true para facilitar el debugging, pero nunca debe pasarse a producción con debug activado.

1.5 PostgreSQL como Base de Datos Segura

PostgreSQL ofrece ventajas de seguridad sobre MySQL en proyectos profesionales: control de acceso por fila (Row Level Security), cifrado nativo, auditoría de transacciones y soporte robusto para tipos de datos. Laravel soporta PostgreSQL de forma nativa a través del driver pdo_pgsql.

■ 2. Procedimiento: Configuración Segura del Proyecto

Paso 1

Crear el Proyecto Laravel

Ejecuta el siguiente comando para crear el proyecto base. Composer descargará e instalará Laravel 12 con todas sus dependencias.

```
# Crear el proyecto en el directorio tasks-app

composer create-project laravel/laravel tasks-app
```

```
# Ingresar al directorio del proyecto

cd tasks-app
```

```
# Verificar la versión de Laravel instalada

php artisan --version
```

> 📋 **Resultado esperado:** Laravel Framework 12.x.x (el número exacto puede variar, pero debe ser versión 12)

Paso 2

Explorar y Comprender la Estructura de Carpetas

Laravel organiza el código siguiendo el patrón MVC. Conocer la estructura es fundamental para ubicar correctamente los controles de seguridad.

```
tasks-app/
├── app/
│   ├── Exceptions/          ← Handler de excepciones (seguridad)
│   ├── Http/
│   │   ├── Controllers/     ← Controladores MVC
│   │   └── Middleware/      ← Filtros de seguridad HTTP
│   ├── Models/              ← Modelos Eloquent
│   └── Providers/           ← Proveedores de servicios
├── bootstrap/
│   └── app.php              ← Punto de entrada, bindings
├── config/
│   ├── app.php              ← Configuración principal (APP_KEY, debug)
│   ├── database.php         ← Configuración de base de datos
│   └── auth.php             ← Configuración de autenticación
├── database/
│   └── migrations/          ← Migraciones de base de datos
├── public/                  ← Único directorio público (Document Root)
│   └── index.php            ← Punto de entrada HTTP
├── resources/               ← Vistas y assets
├── routes/
│   ├── api.php              ← Rutas de la API REST
│   └── web.php              ← Rutas web
├── storage/
│   └── logs/                ← Logs de la aplicación (NO en Git)
├── vendor/                  ← Dependencias Composer (NO en Git)
├── .env                     ← Variables de entorno (NO en Git)
├── .env.example             ← Plantilla del .env (SÍ en Git)
└── .gitignore               ← Exclusiones del repositorio
```

🔐 **Seguridad:** Nunca apuntes el Document Root del servidor web al directorio raíz del proyecto. Solo la carpeta public/ debe ser accesible desde el navegador. Esto evita exposición del código fuente, .env y vendor/.

Paso 3

Configurar PostgreSQL: Crear Base de Datos y Usuario

Antes de configurar Laravel, debemos crear la base de datos y el usuario en PostgreSQL con privilegios mínimos necesarios.

```
# Conectar al servidor PostgreSQL como superusuario
psql -U postgres

-- Crear la base de datos del proyecto
CREATE DATABASE tasks_db
    ENCODING 'UTF8'
    LC_COLLATE 'es_BO.UTF-8'
    LC_CTYPE 'es_BO.UTF-8';

-- Crear usuario con contraseña segura (no usar 'admin' ni '1234')
CREATE USER tasks_user WITH PASSWORD 'T@skS3cur3_2025!';

-- Otorgar solo los privilegios necesarios (principio de mínimo privilegio)
GRANT ALL PRIVILEGES ON DATABASE tasks_db TO tasks_user;

-- Conectar a la base de datos y otorgar privilegios de esquema
\c tasks_db
GRANT ALL ON SCHEMA public TO tasks_user;

-- Verificar la conexión con el nuevo usuario
\q

psql -U tasks_user -d tasks_db -c '\conninfo'
```

⚠️ **Principio de Mínimo Privilegio:** El usuario tasks_user solo tiene acceso a la base de datos tasks_db. No debe tener permisos de superusuario ni acceso a otras bases de datos del servidor.

Paso 4

Configurar el Archivo .env con Variables Obligatorias

El archivo .env contiene toda la configuración sensible del entorno. Copia el archivo de ejemplo y edítalo con tus datos reales.

```
# Copiar el archivo de ejemplo como punto de partida
cp .env.example .env


# Generar la APP_KEY (¡obligatorio antes de cualquier otra cosa!)
php artisan key:generate
```

A continuación, edita el archivo .env con el siguiente contenido completo:

```
# ============================================================
# CONFIGURACIÓN DE LA APLICACIÓN
# ============================================================
APP_NAME="Tasks App"
APP_ENV=local
APP_KEY=base64:GENERADO_POR_ARTISAN_KEY_GENERATE
APP_DEBUG=true
APP_URL=http://localhost:8000


# ============================================================
# REGISTRO Y LOGGING
# ============================================================
LOG_CHANNEL=stack
LOG_DEPRECATIONS_CHANNEL=null
LOG_LEVEL=debug


# ============================================================
# BASE DE DATOS - PostgreSQL
# ============================================================
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=tasks_db
DB_USERNAME=tasks_user
```

```
DB_PASSWORD=T@skS3cur3_2025!


# ============================================================
# CACHÉ Y SESIONES
# ============================================================
BROADCAST_DRIVER=log
CACHE_DRIVER=file
FILESYSTEM_DISK=local
QUEUE_CONNECTION=sync
SESSION_DRIVER=file
SESSION_LIFETIME=120


# ============================================================
# CORREO ELECTRÓNICO (desarrollo)
# ============================================================
MAIL_MAILER=log
MAIL_FROM_ADDRESS="noreply@tasks-app.local"
MAIL_FROM_NAME="${APP_NAME}"
```

✅ **Verificación:** Después de configurar el .env, ejecuta: php artisan config:clear && php artisan config:cache — esto limpia la caché de configuración y detecta errores de sintaxis.

Paso 5

Validar Variables Críticas en config/app.php

Laravel permite validar que ciertas variables de entorno estén presentes al iniciar la aplicación. Abre el archivo config/app.php y verifica las siguientes líneas críticas:

```
<?php


// config/app.php
return [


    /*
     | NOMBRE DE LA APLICACIÓN
     | Nunca uses el nombre por defecto 'Laravel' en producción.
     */
    'name' => env('APP_NAME', 'Laravel'),


    /*
     | ENTORNO DE LA APLICACIÓN
     | Valores válidos: local, staging, production
     | En producción: APP_ENV=production
     */
    'env' => env('APP_ENV', 'production'),


    /*
     | MODO DEBUG - CRÍTICO DE SEGURIDAD
     | NUNCA true en producción.
     */
    'debug' => (bool) env('APP_DEBUG', false),


    /*
     | URL DE LA APLICACIÓN
     | Usada para generar URLs absolutas.
     */
    'url' => env('APP_URL', 'http://localhost'),
```

```
    /*
     | CLAVE DE CIFRADO - OBLIGATORIA
     | Si no está definida, Laravel lanzará excepción al iniciar.
     */
    'key' => env('APP_KEY'),

    'cipher' => 'AES-256-CBC',

    // ... resto de la configuración ...
];
```

Para agregar validación explícita de variables de entorno, añade el siguiente proveedor de servicios. Crea el archivo app/Providers/AppEnvironmentServiceProvider.php:

```
<?php

namespace App\Providers;

use Illuminate\Support\ServiceProvider;
use RuntimeException;

class AppEnvironmentServiceProvider extends ServiceProvider
{
    /**
     * Variables de entorno obligatorias para el funcionamiento
     * seguro de la aplicación. Si alguna falta, la app no inicia.
     */
    protected array $requiredVars = [
        'APP_KEY',
        'APP_ENV',
        'DB_CONNECTION',
        'DB_HOST',
        'DB_DATABASE',
        'DB_USERNAME',
```

```
        'DB_PASSWORD',
    ];


    public function boot(): void
    {
        foreach ($this->requiredVars as $var) {
            if (empty(env($var))) {
                throw new RuntimeException(
                    "Variable de entorno obligatoria no configurada: {$var}"
                );
            }
        }
    }
}
```

Registra el proveedor en bootstrap/app.php:

```
// bootstrap/app.php
->withProviders([
    App\Providers\AppEnvironmentServiceProvider::class,
])
```

Paso 6

Crear un .gitignore Robusto

El .gitignore predeterminado de Laravel es bueno, pero lo reforzaremos para garantizar que ningún archivo sensible llegue al repositorio. Reemplaza el contenido del archivo .gitignore con:

```
# ============================================================
# ARCHIVOS DE ENTORNO Y SECRETOS - NUNCA en el repositorio
# ============================================================
.env
.env.*
!.env.example


# ============================================================
# DEPENDENCIAS - Regeneradas con composer install
# ============================================================
/vendor/
/node_modules/


# ============================================================
# ARCHIVOS GENERADOS Y COMPILADOS
# ============================================================
/public/build/
/public/hot
/public/storage


# ============================================================
# ALMACENAMIENTO Y LOGS - Información sensible de runtime
# ============================================================
/storage/*.key
/storage/logs/*.log
/storage/app/public/


# ============================================================
# CACHÉ DE FRAMEWORK
# ============================================================
```

```
/bootstrap/cache/*.php
/.phpunit.cache

# ============================================================
# ARCHIVOS DE IDE Y SISTEMA OPERATIVO
# ============================================================
.idea/
.vscode/
*.sublime-project
*.sublime-workspace
.DS_Store
Thumbs.db

# ============================================================
# ARCHIVOS DE PRUEBAS Y COBERTURA
# ============================================================
/coverage/
/.phpunit.result.cache
phpunit.xml
!phpunit.xml.dist
```

Verifica que el .env ya rastreado no esté en el índice de Git (si ya fue agregado por error):

```
# Si .env fue agregado por error anteriormente, removerlo del tracking
git rm --cached .env

# Verificar el estado del repositorio
git status

# El .env NO debe aparecer en la lista de archivos rastreados
```

Paso 7

Configurar el Handler de Excepciones para Producción

Laravel 12 centraliza el manejo de excepciones en bootstrap/app.php. Configuraremos el handler para que en producción (APP_DEBUG=false) nunca se revele información interna.

```
<?php


// bootstrap/app.php
use Illuminate\Foundation\Application;
use Illuminate\Foundation\Configuration\Exceptions;
use Illuminate\Foundation\Configuration\Middleware;
use Illuminate\Http\Request;
use Symfony\Component\HttpKernel\Exception\HttpException;


return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        api: __DIR__.'/../routes/api.php',
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware) {
        //
    })
    ->withExceptions(function (Exceptions $exceptions) {


        // Ocultar información sensible en producción
        $exceptions->render(function (\Throwable $e, Request $request) {
            // Solo personalizar respuestas en producción
            if (!config('app.debug')) {


                // Para solicitudes API: respuesta JSON sin detalles
                if ($request->expectsJson()) {
                    $statusCode = $e instanceof HttpException
                        ? $e->getStatusCode()
```

```
                        : 500;

                    return response()->json([
                        'error'   => true,
                        'message' => $statusCode >= 500
                            ? 'Error interno del servidor.'
                            : $e->getMessage(),
                        'code'    => $statusCode,
                    ], $statusCode);
                }
            }

            // En desarrollo, Laravel muestra el stack trace normal
            return null;
        });

        // Reportar excepciones (logging) sin revelarlas al usuario
        $exceptions->report(function (\Throwable $e) {
            // Aquí se puede integrar Sentry, Bugsnag, etc.
            // El log ya es manejado por Laravel automáticamente
        });

    })->create();
```

💡 **Nota:** Con APP_DEBUG=false, Laravel automáticamente muestra páginas de error genéricas (403, 404, 500) sin exponer el stack trace. La configuración del handler agrega control adicional para respuestas JSON de la API.

Paso 8

Configurar PostgreSQL en config/database.php

Aunque la conexión se define en el .env, es buena práctica revisar y ajustar las opciones de PostgreSQL en el archivo de configuración:

```
<?php


// config/database.php - Sección PostgreSQL
'connections' => [


    'pgsql' => [
        'driver'         => 'pgsql',
        'url'            => env('DATABASE_URL'),
        'host'           => env('DB_HOST', '127.0.0.1'),
        'port'           => env('DB_PORT', '5432'),
        'database'       => env('DB_DATABASE', 'tasks_db'),
        'username'       => env('DB_USERNAME', ''),
        'password'       => env('DB_PASSWORD', ''),
        'charset'        => 'utf8',
        'prefix'         => '',
        'prefix_indexes' => true,
        'search_path'    => 'public',
        'sslmode'        => env('DB_SSLMODE', 'prefer'),
        // SSL obligatorio en producción:
        // 'sslmode' => 'require',
    ],


],


'default' => env('DB_CONNECTION', 'pgsql'),
```

Verifica que Laravel puede conectarse a PostgreSQL ejecutando:

```
php artisan db:monitor
# O alternativamente:
php artisan tinker
>>> DB::connection()->getPdo();
```

```
# Resultado esperado: objeto PDO (sin errores de conexión)
```

Paso 9

Crear la Primera Migración: Tabla users

Laravel incluye una migración de users por defecto. La ajustaremos para PostgreSQL y la ejecutaremos para verificar que la conexión funciona correctamente.

Revisa el archivo database/migrations/xxxx_xx_xx_000000_create_users_table.php:

```php
<?php


use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;


return new class extends Migration
{
    /**
     * Crear la tabla de usuarios del sistema tasks-app.
     * Compatible con PostgreSQL y MySQL.
     */
    public function up(): void
    {
        Schema::create('users', function (Blueprint $table) {
            // UUID como clave primaria (más seguro que auto-increment)
            $table->uuid('id')->primary();


            // Datos del usuario
            $table->string('name');
            $table->string('email')->unique();
            $table->timestamp('email_verified_at')->nullable();
            $table->string('password');


            // Token para recordar sesión (hashed)
            $table->rememberToken();


            // Timestamps de auditoría
            $table->timestamps();
```

```
            $table->softDeletes(); // Para no eliminar usuarios permanentemente
        });
    }

    /**
     * Revertir la migración.
     */
    public function down(): void
    {
        Schema::dropIfExists('users');
    }
};
```

Actualiza el modelo User en app/Models/User.php para soportar UUID:

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Concerns\HasUuids;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\SoftDeletes;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

class User extends Authenticatable
{
    use HasFactory, Notifiable, HasUuids, SoftDeletes;

    /**
     * Los atributos que se pueden asignar masivamente.
     * Solo los campos necesarios (no el password ni campos internos).
     */
    protected $fillable = [
```

```
        'name',
        'email',
        'password',
    ];

    /**
     * Atributos que nunca se serializan en JSON.
     * Protege contra exposición accidental de datos sensibles.
     */
    protected $hidden = [
        'password',
        'remember_token',
        'deleted_at',
    ];

    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password'          => 'hashed',
        ];
    }
}
```

Ejecuta la migración:

```
# Ejecutar todas las migraciones pendientes
php artisan migrate

# Verificar las tablas creadas en PostgreSQL
psql -U tasks_user -d tasks_db -c '\dt'

# Resultado esperado:
#  Schema |       Name        | Type  |    Owner
```

```
#  public | migrations        | table | tasks_user
#  public | users             | table | tasks_user
```

Paso 10

Verificación Final del Entorno Seguro

Realiza las siguientes verificaciones para confirmar que el proyecto está correctamente configurado:

```
# 1. Verificar que el servidor de desarrollo funciona
php artisan serve
# Abrir: http://localhost:8000

# 2. Verificar la configuración de seguridad
php artisan about

# 3. Verificar que .env NO está en el tracking de Git
git status
# .env NO debe aparecer

# 4. Verificar que las variables críticas están cargadas
php artisan tinker
>>> config('app.key');   // Debe mostrar la clave (no vacío)
>>> config('app.debug'); // Debe ser true (en desarrollo)
>>> config('app.env');   // Debe ser 'local'
>>> DB::connection()->getPdo(); // Debe retornar objeto PDO

# 5. Probar el modo producción (APP_DEBUG=false)
# Temporalmente cambiar en .env: APP_DEBUG=false
php artisan config:clear
# Provocar un error: visitar una ruta inexistente
# Verificar que solo se muestra '404 Not Found' sin stack trace
# Restaurar: APP_DEBUG=true
```

✅ **Estado esperado al finalizar:** El servidor Laravel corre en localhost:8000, la base de datos PostgreSQL está conectada, la tabla users existe, el .env no está en Git, y la APP_KEY está generada. El proyecto está listo para las siguientes guías.

■ 3. Capturas de Pantalla Esperadas

A continuación se describe el resultado visual esperado en cada etapa clave. El estudiante debe documentar estas capturas en su reporte de laboratorio.

#	Momento	Descripción del resultado esperado
1	**composer create-project**	La terminal muestra la descarga de paquetes con barras de progreso y al finalizar imprime 'Application key set successfully.'
2	**php artisan --version**	Salida: 'Laravel Framework 12.x.x' indicando instalación exitosa de la versión 12.
3	**psql: \dt en tasks_db**	Listado de tablas que muestra al menos 'migrations' y 'users' en el esquema public, propietario tasks_user.
4	**php artisan migrate**	La terminal muestra ' INFO Running migrations.' seguido de '... DONE' para cada migración ejecutada sin errores.
5	**php artisan serve**	Terminal muestra 'INFO Server running on [http://127.0.0.1:8000]'. El navegador abre la pantalla de bienvenida de Laravel.
6	**php artisan about**	Panel informativo de la app: muestra el entorno, versión de PHP, conexión a base de datos y estado de la caché.
7	**git status (sin .env)**	El archivo .env NO aparece en la sección 'Untracked files'. Solo se listan archivos que SÍ deben estar en el repositorio.
8	**APP_DEBUG=false (error 404)**	Al visitar una ruta inexistente con debug en false, se muestra una página de error genérica sin stack trace ni información del servidor.

📸 **Instrucción:** Toma capturas de pantalla reales de tu terminal y navegador al completar cada paso. Las capturas deben incluir tu nombre de usuario del sistema operativo visible en el prompt de la terminal para validar autenticidad.

■ 4. Rúbrica de Evaluación

Puntuación total: 33 puntos. La nota se convierte a escala 0–100 para el registro académico.

#	Criterio de Evaluación	Indicador de Logro	Pts.	Obtenido
1	**Creación del proyecto con Composer**	El proyecto tasks-app se creó correctamente con composer create-project y la versión de Laravel es 11.x	3	
2	**Generación y validación de APP_KEY**	La APP_KEY está generada con php artisan key:generate y tiene formato base64:xxxxx... (44 chars)	3	
3	**Configuración de .env con PostgreSQL**	El .env contiene todas las variables DB_ correctas y la conexión a PostgreSQL es exitosa (db:monitor muestra OK)	4	
4	**AppEnvironmentServiceProvider implementado**	El proveedor valida variables obligatorias y está registrado en bootstrap/app.php. Lanzar error si falta DB_PASSWORD.	3	
5	**.gitignore robusto**	El .env no aparece en git status como archivo rastreado. El .gitignore excluye .env, vendor/, storage/logs/	3	
6	**Handler de excepciones configurado**	Con APP_DEBUG=false, una ruta inválida muestra error genérico sin stack trace. La captura de pantalla lo evidencia.	4	
7	**Migración ejecutada correctamente**	php artisan migrate crea las tablas migrations y users en tasks_db. psql \dt confirma su existencia.	4	
8	**Modelo User con UUID y $hidden**	El modelo usa HasUuids, el campo password está en $hidden, y softDeletes está activo.	3	
9	**Documentación: Capturas de pantalla**	El reporte incluye las 8 capturas descritas en la Sección 3, con el prompt del usuario visible.	3	
10	**Reporte y conclusiones**	El reporte incluye análisis de por qué APP_DEBUG=false es crítico en producción y qué riesgos mitiga el .gitignore.	3	
		TOTAL	**33**	

Escala de Calificación

Rango de Puntos	Calificación	Nota (escala 100)	Nivel de Logro
30 – 33	Excelente	91 – 100	Supera las expectativas
25 – 29	Bueno	76 – 90	Cumple completamente
18 – 24	Regular	55 – 75	Cumple parcialmente
0 – 17	Insuficiente	0 – 54	No cumple los mínimos

Lista de Verificación para Entrega

✓	Ítem a entregar	Entregado
☐	Repositorio Git con el proyecto tasks-app (sin .env, sin vendor/)	
☐	Archivo .gitignore robusto visible en el repositorio	
☐	Archivo .env.example con todas las variables (sin valores reales)	
☐	AppEnvironmentServiceProvider.php implementado y registrado	
☐	Migración ejecutada y tablas visibles en PostgreSQL	
☐	Modelo User con UUID, $hidden y SoftDeletes	
☐	Reporte PDF con las 8 capturas de pantalla documentadas	
☐	Conclusiones: análisis de riesgos mitigados por la configuración segura	

📅 **Fecha de Entrega:** La guía debe entregarse hasta la siguiente sesión de laboratorio.